CONFIDENTIAL — INVESTOR TECHNICAL BRIEF

Platform evaluation — Concept2Cure.RI / TrialSage

Version · 2026-09-10 · rev 2
Codebase branch · `concept2cure-v2` @ `d31a9db6` (measured), remediation through `27c3d16d`
Audience · owner, engineering leads, and anyone deciding whether to put real customer data in front of a human tester

PREPARED FOR PROSPECTIVE INVESTORS

How to read this

Every figure below was produced by a command run against this repository at `d31a9db6` on Node 22.22.2 with a clean `npm ci`. Nothing is inherited from a prior assessment without being re-executed, and where a prior figure is quoted for comparison it is labelled with its source and SHA.

Results are reported in five classes and never collapsed into "pass":

CLASS	MEANING
ZERO-DEBT PASS	Gate has no baseline. Clean.
RATCHET PASS — N REMAIN	Gate passed because the count is no worse than a baseline of N. **N defects still exist.**
INVENTORY	Script always exits 0. Cannot pass or fail. Not evidence of anything.
FAIL	Gate found something above its threshold.
ENV-BLOCKED	Could not run here (needs a live database, network, or a build artifact). Not a pass.

Rev 2 (2026-09-10, same day). This document was written, then acted on, and the acting changed it. Work orders WO-0 and WO-4 were executed and closed, WO-1 started, and two live defects were found and fixed that no gate had reported (§1). Four of my own findings turned out to be wrong and are struck through in place rather than deleted — a retracted finding is evidence about how the finding was reached, and deleting it would leave the document looking more reliable than the process that produced it. Where a figure moved, both the old and new value are shown.

There is no composite score in this document. A "6.5 out of 10" is not derivable from anything measured here, and a number like that gets repeated in rooms where its provenance cannot be checked. The readiness question is asked instead against the G1/G2/G3 ladder this repository already reasons in (`docs/audit-2026-07/14-readiness-gate-ladder.md`), because a verdict that attaches to an existing ladder can be argued with.

---

1. Executive summary

The platform is materially better than it was six weeks ago, and the headline risk has moved. Between `576ec5d` (2026-07-28, the last purchase-grade audit) and `d31a9db6` there are **918 commits**. Measured against that audit's own suppression basket, most tranches went *down*, several substantially:

SUPPRESSED DEFECT CLASS	2026-07-28 `576EC5D`	2026-09-10 `D31A9DB6`	Δ
Tenant-isolation raw-SQL candidates	25	**10**	**−15**
Unbacked tables (queried, never created)	89	**36**	**−53**
Unreferenced modules	190	**98**	**−92**
Mock/simulated markers in production routes	11	**0**	**−11**
Route-mount errors	8 errors + 7 warnings	**0 errors** + 8 warnings	**−8 errors**
Orphan endpoint candidates	556 of 914	**505 of 884**	−51
Duplicate-basename groups (repo health)	249 groups	**0**	**−249**
Files on the shared pool	81 †	**229** †	*not comparable — see §5.1*
Files bypassing the governed AI gateway	**3**	**19**	**+16**

† Both figures come from the same gate, and the gate was miscounting. Its detector matched one of the nine import shapes these routes use, so the true count on this row was never 81 or 82. Widened and rebaselined to **229** during this evaluation; the

July figure cannot be restated, so the delta is unknown rather than +1. §5.1 has the mechanism, and it is less alarming than the number looks.

Four of that audit's seven G1 blockers are demonstrably fixed at HEAD, and fixed properly rather than papered over — §6 shows the code.

What has not moved is the thing that now matters most. Schema authority is unresolved and is upstream of every assurance claim the pilot depends on:

- **64 tables have more than one `CREATE TABLE` definition** across 593

non-archived SQL files — one more than the baseline of 63, so `ci:duplicate-table-ddl:strict` fails on an unbaselined regression.

- **73 tables the server queries do not exist on a live database**

(`ci:tables-live-schema` , baseline 73 — a green gate).

- **36 tables are referenced by runtime code and created by nothing in the repo.**
- **4 migration prefixes collide** across 13 files.

Combined with `CLAUDE.md` RULE 1 — every migration re-runs on every deploy, unconditionally, and journal drift is written but never read — this produces a specific, checkable claim, and it is the most important sentence in this document:

> **The schema of a deployed environment is not derivable from the repository.**

`scripts/ci/check-duplicate-table-ddl.mjs` says so itself: because every definition is `IF NOT EXISTS`-guarded, "the surviving column set is decided by migration ORDER, not by code — and it can differ per environment." For a platform whose value proposition is regulatory record-keeping, that is not maintainability debt. It means RLS coverage, purge proof, and audit reconciliation are all assertions against a target no one can pin down.

Verdict against the ladder:

GATE	VERDICT	DISTANCE
G1 · Pilot on non-regulated data	🟡 **CLOSE**	1–2 weeks — WO-4, WO-6, WO-9
G1+ · Pilot on real customer data ← *your stated bar*	🔴 **NOT READY**	**4–7 weeks** — WO-1, WO-2, WO-3 are hard prerequisites
G3 · GxP / submission-grade	🔴 **NOT READY**	6–12 months, mostly non-engineering

Your stated bar does not sit on a rung of the existing ladder. That audit defines G1 as design partners on *non-regulated* data, and explicitly carves out tenant isolation: "tenant isolation being single-layer (RLS inert) is acceptable for a non-regulated pilot with a handful of trusted design partners." **That carve-out expires the moment the data is real.** An external pilot on real customer data is G1 plus the tenancy half of G2, and the work orders in §7 are scoped to exactly that.

The finding that outranks all of the above

`concept2cure-v2` is red right now on six gates that CI actually runs. Not strict variants that run nowhere — gates wired into `ci.yml` and `pr-checks.yml` :

GATE	BASELINE	MEASURED	DELTA
`ci:eslint-ratchet`	6,596	**6,712**	+116 warnings, and 1 error
`ci:check-phantom-tokens`	14	**17** (66 sites)	+3
`ci:duplicate-table-ddl`	63	**64**	+1 table with a second definition
`ci:model-migration-agreement`	29	30	+1 table diverging from its migration
`ci:tenant-blind-models`	5	—	stale baseline — an entry got *fixed* and was never removed
`ci:tenant-entry-points`	10	—	`retentionCron.ts` changed after its entitlement justification

Four of those six are regressions a working pipeline should have refused at the PR that introduced them. That they sit on the branch `CLAUDE.md` RULE 0 designates as the only branch that ships means either CI is not gating merges to it or red results are being merged past. **That is worth resolving before planning a customer pilot around this pipeline**, and it is WO-0.

The other two are instructive rather than alarming: `tenant-blind-models` fails because a baselined defect was *fixed* and nobody removed the entry. The gate enforces bidirectional parity — a stale baseline fails as loudly as a new defect, because a baseline that overstates debt hides the next real one. That is good design, and it is the design WO-5 proposes extending to the other 42 baselines.

> **RESOLVED 2026-09-10, same day.** WO-0 was executed against this finding; all > six now pass and the sweep's non-zero count fell 28 → 23. Three of the six > were not what the gate reported: `model-migration-agreement` was a **parser > bug** (its `ALTER TABLE` regex saw only the first `ADD COLUMN` of a > comma-separated statement, so the "fix" it demanded would have added a > migration for columns that already existed); `duplicate-table-ddl` was a > genuine **two-applier schema split** on `cmc_comparability_assessments`, which > is §4's thesis in miniature; and `tenant-blind-models` was a stale baseline > over an already-fixed cross-tenant leak. One caveat survives: the eslint total > is green because 130 unused imports were removed, **not** because the > complexity growth was fixed — `complexity` +34 and `max-lines-per-function` > +34 remain inside the new 6,575 baseline. See > [`WO-0`](../work-orders/WO-0-restore-green-canonical-branch.md) for the full > outcome.

Two live defects, found by execution, that no gate found

Everything above came from gates. The two most serious findings in this evaluation did not, and that is the most useful thing in the document.

1. Any tenant could read any other tenant's Part 11 audit trail. `GET /api/grdhe/audit/:tableName/:recordId` was mounted behind `authenticateToken` and nothing else. It took a table name and a record id, checked the table name against an allowlist — an anti-SQL-injection measure, not an authorization one — and returned the full audit history for that record, including the `old_data` and `new_data` JSONB columns, which hold the before and after of every regulated change. There was no tenant predicate in the query, no tenant column on the audit table to write one against, and no RLS policy: the enforcement sweep scans `WHERE c.table_schema = 'public'` and these tables are not in `public`. Record ids are sequential. Any authenticated user of any tenant could enumerate them and read every other tenant's regulated change history.

Fixed in `5aa07bc8e`. Every auditable table is now declared as `tenant`-scoped, `global`, or `unscopable`; a `tenant` table gets an EXISTS probe against the caller's org before the audit query runs, and an unlisted table is refused rather than served. A cross-tenant request returns an empty result rather than 403, deliberately — a 403 confirms the record exists, which is the same leak in a smaller quantity. **WO-13** carries the remaining half: `electronic_signatures` still has no tenant column to scope by, so it is currently declared `unscopable` and refused outright — and §5.3 finds the same hole in a second, unrelated subsystem.

2. Two fabricated CAPA records were inserted on every deploy. `db/migrations/030_stability_results.sql` ended in an unguarded `INSERT INTO capa` — no `WHERE NOT EXISTS`, no `ON CONFLICT`, and `capa.id` is `SERIAL`, so nothing could dedupe it. Under RULE 1 that file replays on every deploy, so an estate that has deployed *N* times carries *2N* of these. CAPA is a corrective-and-preventive-action record. One of the two names an investigator, "Dr. Johnson", who does not exist.

The INSERT is gone (`cdd12788c`), which does not remove rows already written; `npm run ops:audit-fabricated-capa` (`e650c5e3d`) reports them per environment and deletes only rows still byte-identical to the seed, never one a human has since edited.

What these two have in common is more important than either. Neither is subtle, and 142 gates missed both — because both live in the space the gates do not model. The tenancy gates count *files* on the shared pool and *raw SQL without a tenant predicate*; grdhe's query had a predicate (on `record_id`), and a table with no tenant column cannot appear in a sweep that looks for tenant columns, so it was invisible to every one of them. RULE 1's drop-safety gate checks that migrations do not DROP; no gate checks that a replayed migration does not INSERT.

A third defect, found the same way and after those two, makes the point sharper. `auditService.getAuditLog` applied its tenant filter on the primary path and, when that path failed, fell through to a store with no tenant column — returning every tenant's rows as an array the caller could not distinguish from a correct answer. Nothing reaches it today, so it is a refusal now rather than an incident. Three findings, three unrelated subsystems, one shape: **the tenant isolation is on the data and not on the record of what happened to the data** (§5.3). A suppression ledger of 9,053 measures the debt the gates can see. It says nothing about the debt they cannot, and this evaluation found two of those in a week of reading — which is the strongest argument in the document for WO-3's live probe over any amount of additional static analysis.

The three things that decide the pilot bar

1. **Schema authority** (WO-1, WO-2). Until one manifest defines each table

once and a blank database provisions everything the server queries, no isolation or retention proof means anything.

1. **Tenant isolation proven, not asserted** (WO-3). The 10 raw-SQL candidates

are static analysis, and 229 route files still run on the shared pool (§5.1 — instrumented, so this is depth rather than absence). Neither is a proof. The proof is a live two-tenant probe, and none has ever been run.

1. **Nothing drives the debt to zero** (WO-4). Six `:strict` variants — the

ones that demand an *empty* baseline — are in `package.json` and in no pipeline. ~~Every finding in §1 could regress tomorrow without CI noticing.~~ **Corrected 2026-09-10:** that was wrong. All six *non-strict* variants run in `ci.yml`'s `lint` job on every push and PR, and they do catch growth — the non-strict `ci:duplicate-table-ddl` is what caught WO-0's 64th table. The debt cannot grow; nothing makes it shrink, and nothing reported how far from zero it was. Fixed by running the six `:strict` gates nightly.

---

2. Method, and what this evaluation cannot tell you

Executed here: the full `ci:` / `audit:` / `readiness:` inventory (145 scripts after excluding `:write-baseline` , `:list` , and three destructive or network-bound entries), the baseline census, and targeted re-tests of the seven G1 blockers from the July audit. The runner is `evidence/sweep.mjs` ; raw results are `evidence/01-gate-sweep.json` . The ledger census is `evidence/ledger.mjs` → `evidence/02-suppression-ledger.json` .

Not executed, and therefore not claimed:

- **The test suite.** 464 test files exist. None were run. A DB-backed suite

would fail here on environment, and reporting that as a finding would repeat the error this document exists to correct.

- **Anything requiring a live database.** `ci:tables-live-schema` and

`ci:purge-coverage` are **ENV-BLOCKED**. Their baselines are read from disk and reported as baselines, not as passes.

- **Anything requiring the network.** `ci:dependency-risk` needs the npm

advisory API; it returned HTTP 403 through this environment's proxy.

- **Runtime behaviour.** No browser, no boot, no request was issued. Every

tenancy finding below is static analysis, and is labelled as such.

One methodological correction to the assessment this replaces. A prior evaluation of this platform was produced on 2026-09-09 as a deliberate "clean-room" exercise, "without consulting prior platform assessments." Its gate numbers are accurate — I re-ran its six headline figures and matched all six exactly. But the clean-room premise cost it the trend, and the trend is the finding: a snapshot of 10 tenant-isolation candidates reads as an alarm, while 25 → 10 over 918 commits reads as a burndown working. That evaluation also reported its own commits, SHAs, and pull requests as having landed on `concept2cure-v2`. None had; its sandbox removes the `origin` remote at bootstrap, so the work could not leave the container. **Its observations were sound and its self-reporting was not**, which is a useful reminder that a document's provenance is part of its evidence.

2b. Gate truth — all 142 gates

`evidence/sweep.mjs` ran every `ci:` / `audit:` / `readiness:` script except `:write-baseline` (rewrites the thing being measured), `:list` (always exits 0), `audit:prune-artifacts` (deletes files), `audit:last-20-prs` (runs `npm ci` and queries GitHub) and `audit:bundle` (full production build).

142 gates ran. 114 exited 0. 28 did not. Those 28 are not 28 failures:

CLASS	COUNT	GATES
FAIL — CI-enforced	**6**	`eslint-ratchet`, `check-phantom-tokens`, `duplicate-table-ddl`, `model-migration-agreement`, `tenant-blind-models`, `tenant-entry-points` → **WO-0**
FAIL — strict variant, enforced nowhere	6	`tenant-isolation:strict`, `duplicate-table-ddl:strict`, `unbacked-tables:strict`, `unreferenced-modules:strict`, `migration-prefix-collisions:strict`, `reasoning-tier-*:strict` → **WO-4**
FAIL — real, not CI-wired	6	`tenant-resolvers` (190 baselined, above it), `duplicate-exported-types`, `surface-text-ramp`, `audit:repo-health:strict` / `:full-strict` (42 files >100 KB vs ceiling 33; 95 >1,500 lines vs 82)
ENV-BLOCKED — needs a live database	5	`tables-live-schema`, `purge-coverage`, `audit:archive`, `audit:verify:24h`, `audit:verify:full`, `readiness:check:strict`
ENV-BLOCKED — needs network	2	`dependency-risk`, `dependency-risk:reseal` (npm advisory API returned 403 through this proxy)
ENV-BLOCKED — needs a build or coverage artefact	2	`component-class-coverage` (**exit 2** = stale/absent build, which the script distinguishes from exit 1 = real findings), `coverage-ratchet`

And the 114 that exited 0 are not 114 clean bills of health. Of them, at least 31 are ratchet passes standing on a non-empty baseline (§3), and a further set cannot fail at all. `readiness:check` non-strict prints `WARN-ONLY` in its own header; all 14 `:list` variants, `ci:report-branch-drift` and `audit:evidence-pack` exit 0 unconditionally; `ci:token-cascade` is `continue-on-error: true` in `ci.yml:590` and cannot block a merge regardless of result.

Raw results, with exit code and output tail per gate: [`evidence/01-gate-sweep.json`](evidence/01-gate-sweep.json).

---

3. The suppression ledger

Baseline files are the most honest artefact in this repository, and there are now **43** of them. A gate that reads a baseline cannot report "clean"; it reports "no worse than N". Summed exactly (`evidence/ledger.mjs` , explicit per-file extractors — an earlier heuristic version mis-read six of them):

CLASS	ENTRIES
Defect-class entries under active suppression	**2,371**
Lint / cosmetic entries	6,682
Total tolerated across 43 baselines	**9,053**

Re-measured 2026-09-10 after this evaluation's own changes: **+127 defect-class** (the `requestdb-coverage` detector fix, 82 → 229, which found no new debt — it stopped failing to see debt that was already there) and **−21 lint** (the unused imports removed under WO-0). An earlier draft of this table read 2,244 / 6,703 / 8,947.

Excluded from the total, because neither is a count of tolerated defects: `coverage-baseline.json` (a percentage floor) and `proof-tier-baseline.json` (an **inverted** ratchet — a floor of 77 proof files that must not disappear, i.e. an asset).

The ten largest tranches

ENTRIES	BASELINE	WHAT IT TOLERATES
6,575	`eslint-warning-baseline.json`	Lint warnings across 21 rules
611	`purge-coverage-baseline.json`	Tables with no proven purge path
246	`duplicate-exported-types-baseline.json`	Exported names meaning two different things
244	`env-var-docs-baseline.json`	Env vars read by code, absent from `.env.example`
229	`requestdb-coverage-baseline.json`	Route files still on the shared pool — **82 until the detector was fixed on 2026-09-10** (§5.1)
190	`tenant-resolvers-baseline.json`	Modules not migrated to the canonical tenant resolver
151	`drizzle-tenant-scope-baseline.json`	ORM query sites with no tenant scope
151	`server-error-leaks-baseline.json`	Sites leaking internal error detail (92 files)
98	`unreferenced-modules-baseline.json`	Modules nothing references
80	`dead-audit-tables-baseline.json`	Audit tables nothing writes to

Reading the growth honestly

The July audit put the comparable figure at ~1,620; the defect-class figure is now 2,371. **That is not 750 new defects.** Most of the difference is *new instrumentation finding pre-existing debt*: `tenant-resolvers` , `drizzle-tenant-scope` , `server-error-leaks` , `dead-audit-tables` , `tables-live-schema` , `insert-columns` , `model-migration-agreement` and `writerless-stores` are gates that did not exist in July. A gate landing with a baseline of 151 has discovered 151 defects, not created them.

This distinction matters for how the ledger is read as a signal. **Gates appearing is good news that looks like bad news.** The genuinely bad news is narrower and worth naming precisely:

- **`gateway-bypass` 3 → 19.** Nineteen files reach a model provider outside the

governed AI gateway — including two `.js` shadow files (`semanticSearch.js` , `huggingface-service.js`). Each entry carries a written reason naming what governance is lost. On a platform whose differentiator is a governed AI layer, sold to regulated customers, this is the regression that should worry you most after schema authority.

- `duplicate-table-ddl` **63 baselined, 64 measured.** One unbaselined

regression — a table gained a second definition and no gate in any pipeline caught it, because `:strict` runs nowhere (§5).

- `env-var-docs` **244, unchanged, and still wired to no npm script.** The July

audit flagged this gate as non-existent. It is still non-existent.

---

4. Schema authority — the blocking risk

4.1 The mechanism

Three facts, each independently verified, combine into one problem.

Fact 1 — every migration re-runs on every deploy. `applyMigrationFiles` (`scripts/db/migration-set.mjs`) executes every entry of `C2C_MIGRATION_FILES` unconditionally. There is no "already applied, skip" branch. `recordApplied` computes a content hash and returns `'new' | 'unchanged' | 'drift'`, and the caller discards it. Drift is written to `c2c_migration_journal`; nothing reads it. This is documented as RULE 1 in `CLAUDE.md` and is by design — the set is replayable because nearly every file is additive and `IF NOT EXISTS`-guarded.

Fact 2 — 64 tables have multiple definitions. Measured:

``` $ npm run ci:duplicate-table-ddl:strict [ci:duplicate-table-ddl] scanned 593 non-archived .sql files; 1186 distinct tables; 64 defined more than once (strict mode) ```

Fact 3 — the guard that makes replay safe is what makes definitions ambiguous. From the gate's own output: because each definition is `CREATE TABLE IF NOT EXISTS`, "the surviving column set is decided by migration ORDER, not by code — and it can differ per environment."

4.2 The consequence

Put together: **the deployed schema is a function of file ordering across multiple migration roots, not of the repository's contents.** Two environments that applied the same commit in a different order can hold different column sets, and nothing reports the difference — the journal records drift that nothing reads.

Downstream, this makes several assurance claims unfalsifiable rather than false:

- **RLS coverage** asserts policies against tables whose columns are

order-dependent.

- `ci:purge-coverage` tolerates 611 tables with no proven purge path — a

retention claim over a schema that cannot be pinned down.

- **Audit reconciliation** compares a chain to tables that may or may not have

the columns the code expects.

- `ci:tables-live-schema` already measures the gap directly: **73 tables the

server queries do not exist on a live database.** That gate is green, because 73 is its baseline.

For a pilot on real customer data, "we cannot state what schema is deployed" is a stop condition, not a caveat. It is WO-1 and WO-2, and they are upstream of everything else.

4.3 What is *not* wrong here

The migration system is not carelessly built. `ci:migration-drop-safety` , `ci:migration-set-order` and `ci:migration-reachability` all exist and pass, ADR-0006 defines canonical lineage, and the drop-safety gate ships with
```

a self-test that constructs the real create-then-drop hazard in both orders. The problem is not absence of discipline; it is that the discipline has not yet been applied to the 64 pre-existing collisions, and the gate that would hold the line runs in no pipeline.

---

5. Tenancy — and the gates that guard nothing

5.1 Measured state

``` $ npm run ci:tenant-isolation:strict Total: 10 candidate(s). These are RAW SQL queries against known tenant-scoped tables that don't reference an org_id / tenant_id / workspace_id in the same statement. ```

Ten, down from 25 in July. In context, though, the raw-SQL candidates are the smallest of five overlapping tranches:

ENTRIES	BASELINE	MEANING
190	`tenant-resolvers`	Modules not on the canonical tenant resolver
151	`drizzle-tenant-scope`	ORM query sites with no tenant scope
229	`requestdb-coverage`	**Route files still on the shared pool** — corrected from 82 below
10	`tenant-isolation`	Raw SQL with no tenant predicate
10	`tenant-entry-points`	Entry points whose tenant entitlement drifted
5	`tenant-blind-models`	Models with no tenant column at all

~~The 82 shared-pool route files are the load-bearing number. RLS policies exist, but a route on the shared pool connects as a role that bypasses them, so the second layer of defence is not merely weak — for those routes it is not engaged.~~

Corrected 2026-09-10. Both halves of that were wrong.

The count. It is **229**, not 82. `ci:requestdb-coverage`'s detector recognised one of the nine import shapes a route uses — it required the binding to be literally `db`, in braces, from a specifier with no extension — so it was blind to 147 files. Worse, the ratchet could be moved for free: this evaluation's own WO-0 lint cleanup rewrote `import { db, pool }` to `import { pool }` in `tenant-section-gating.ts`, and the gate stopped counting a file that still runs eight `pool.query()` calls on the shared pool. Detector widened and rebaselined to 229, with the reason recorded in the baseline file.

The mechanism. A shared-pool route does **not** simply bypass RLS. `server/db/poolInstrumentation.ts` wraps the pool so that under `RLS_ENFORCE=on` every non-infrastructure query runs inside a micro-transaction that applies the tenant scope first, and **fails closed** with no scope; `server/db/rlsEnforcement.ts` refuses to boot production on anything else. The second layer *is* engaged.

The migration still matters, for narrower and more specific reasons:

- **No read consistency.** Per-statement micro-transactions, not a pinned

connection — a read-then-write pair in one handler can straddle another tenant's commit.

- **The protection is ambient, not lexical.** It depends on AsyncLocalStorage

scope surviving every async boundary — timers, emitters, queue callbacks. `requestDb(req)` makes the dependency an argument you can see.

- **Module-level singletons escape it entirely** — a pool captured at import time

runs outside any request.

- **One env var is the whole layer.** `RLS_ENFORCE ≠ 'on'` makes the
```

instrumentation inert.

So this is defence-in-depth, not an open door — which is the opposite of §5.3 below, where RLS enforcement genuinely does not help.

5.2 The finding that generalises

Six `:strict` gates run in no pipeline — the variants that demand an empty baseline rather than "no worse than N". Cross-referencing `package.json` against `.husky/pre-push`, `.github/workflows/ci.yml` and `.github/workflows/pr-checks.yml`:

GATE	IN CI?
`ci:tenant-isolation:strict`	❌ nowhere (only `:no-regression` runs)
`ci:duplicate-table-ddl:strict`	❌ nowhere
`ci:unbacked-tables:strict`	❌ nowhere
`ci:unreferenced-modules:strict`	❌ nowhere
`ci:migration-prefix-collisions:strict`	❌ nowhere
`ci:proof-tier:strict`	❌ nowhere

Corrected 2026-09-10 (WO-4). An earlier draft concluded from this table that "every one of §1's findings is free to regress without CI noticing." That does not follow, and it is wrong. Each of the six has a *non-strict* sibling running in `ci.yml`'s `lint` job — no `if:` condition, so every push and PR — and those siblings do catch growth. The regression this section cited as proof (`duplicate-table-ddl`, 63 baselined vs 64 measured) was caught **by exactly that non-strict gate**, which is how WO-0 found it.

What the missing `:strict` variants actually cost is narrower and still real: the debt is prevented from growing but nothing drives it to zero, and no run reported how far each baseline was from empty. WO-4 put all six into the nightly governance job so that distance is now published daily.

Three further honesty defects in the gate layer, each of which makes a green build mean less than it appears:

- **`ci:token-cascade` is `continue-on-error: true`** (`ci.yml:590`). Its

pass/fail cannot block a merge.

- **`ci:no-dev-auth-in-prod` non-strict is not lenient.**

`check-no-dev-auth-in-prod.mjs:205` reads `process.exit(strict ? 1 : 1)` — the two modes are identical.

- ** `ci:fixture-fallback` and `ci:org-path-param-guards` read baseline files

that do not exist on disk**, so they currently behave as zero-debt gates by accident rather than by decision.

5.3 The platform policies its data and not its audit trails

This section replaces a draft written earlier today that was wrong, and the error is worth stating because it is the same error three times. That draft said RLS covers only schema `public`, and named `regulatory_harmonization.*` as a table the sweep therefore misses. Both halves are false.
`db/migrations/20260801_uuid_tenant_isolation_nonpublic.sql` is a second sweep built for exactly this: an explicit 28-entry `(schema, table, column)` list covering `core`, `manufacturing`, `compliance`, `global_dossier`, `cortex`, `federated_ml`, `regulatory_intel` and `regulatory_harmonization`, keyed per-table because the tenant column name differs (`org_id` / `organization_id` / `tenant_id`).

`canonical_adverse_events` and `canonical_products` — the patient-level tables — are in that list, on `tenant_id`. They are policied.

The right question was never "which schema". It is **which tables carry a tenant key at all**, and the answer separates cleanly along a line nobody drew on purpose:

	TENANT COLUMN	POLICIED
`regulatory_harmonization.canonical_adverse_events`	`tenant_id`	✅
`regulatory_harmonization.canonical_products`	`tenant_id`	✅
`regulatory_harmonization.export_jobs`	`tenant_id`	✅
`regulatory_harmonization.tenant_data_residency`	`tenant_id`	✅
`regulatory_harmonization.audit_log`	**none**	❌ *cannot be*
`audit.tamper_proof_log`	**none**	❌ *cannot be*
`regulatory_harmonization.electronic_signatures`	**none**	❌ *cannot be*

The data is isolated. The record of what happened to the data is not. Both subsystems reached that state independently, which makes it a pattern rather than an oversight: `audit_log` carries `user_organization TEXT` — free-text metadata about who acted, not a key anything can filter on — and `tamper_proof_log` carries nothing at all (id, sequence, event type, actor, resource, action, details, hash chain, client context).

And the 2026-08-01 sweep did think about this. It exempted `audit.event_log` deliberately, in writing:

*> 21 CFR Part 11 immutable audit trail, written by DB triggers … and read only > by cross-org compliance views and a background hash-integrity job; **there is > no per-tenant request reader to isolate**, and a policy would drop NULL-org > system events and break the compliance/export readers.*

That is the right way to make the call, and for that table the reasoning holds. `regulatory_harmonization.audit_log` never reached the same triage — and unlike `audit.event_log`, it **did** have a per-tenant request reader: `GET /api/grdhe/audit/:tableName/:recordId`, mounted on `authenticateToken`, serving `old_data` / `new_data` for any record id a caller could guess. So the gap was not a missing sweep. It was one audit table that got the analysis and one that did not.

That also explains why the fix (§1) took the shape it did. With no tenant column on the audit row there is nothing to police, so the guard derives the tenant from the **audited record** instead: an EXISTS probe against the parent table — which *is* policied — before the audit query runs. Same for `auditService.getAuditLog`, where the tamper-proof fallback now refuses a tenant-scoped read outright rather than answering it from a table that cannot honour the scope.

The test that separates a real finding from a false alarm, since I got it wrong in both directions today:

- **`stab_signoffs` looked untenanted and is not.** Its `CREATE TABLE` has no

tenant column, and I wrote it up on that basis alone. Wrong:
`db/migrations/20260728_stability_tenant_isolation.sql` runs later on the same applier and sweeps every `stab_*` table — adding `tenant_id`, enabling *and* forcing RLS, creating `tenant_isolation_policy`. Reading the creating migration is not reading the schema.

- **`regulatory_harmonization` looked unpoliced and is not** — the correction

above. Reading one sweep is not reading the sweeps.

- **`grdhe`'s audit table looked like the rest of its schema and was not.**

So: check for an actual `CREATE POLICY` on the actual table, after every migration on every applier has run. Nothing short of that is evidence.

Applying it to the two readers with the least defence-in-depth: `csr-analytics.ts`'s ten unpredicated reads hit `csr_reports`, which is `public` with an integer `organization_id` and is swept by `0021_enable_rls_everywhere`; `graphrag.ts`'s nine sites hit `knowledge_graph_nodes` / `_edges`, which `db/migrations/20260813_knowledge_graph_tenant_keys.sql` policies explicitly, and whose own comment records the right instinct — *"the policy below makes NULL rows visible to NOBODY rather than to everybody."* Both are defence-in-depth, not open doors.

WO-13 is now scoped to the row that is left: give the two audit trails and `electronic_signatures` a tenant key, or — where a cross-org reader is genuinely required — an exemption written down and reasoned the way `audit.event_log`'s was, so the next per-tenant reader mounted over one of them is a decision rather than an accident.

---

6. The July G1 blockers, re-tested at HEAD

Each of the seven was re-checked in the code, not inherited.

#	JULY FINDING	STATUS AT `D31A9DB6`	EVIDENCE
G1-1	Fresh install silently half-works; skipped migrations reported as success	🟢 **FIXED**	`scripts/db/install-fresh.mjs` now reports skips **by name**; its own header documents the old behaviour ("described, without evidence, as 'safe to skip'") and states skips are "not silently faked"
G1-2	`/readyz` green over a database missing auth tables	🟢 **FIXED**	`server/startup/services.ts` now calls `setSchemaReadiness` on **13** paths including every error and missing-table branch
G1-3	Live cross-tenant write path (Schedule-of-Events)	🟡 **PARTIAL**	Raw-SQL candidates 25 → 10, and 229 route files remain on the shared pool — instrumented, so protected in depth (§5.1). Still static only: no live probe has ever been run. **WO-3**
G1-4	Stored XSS via `derivePreview` HTML-entity decode	🟢 **FIXED**	`derivePreview` strips tags, decodes only safe entities, then strips any residual `[<>]`, and documents "The result is TEXT." `BatchDraft.tsx` now renders through a DOMPurify allowlist; the one remaining `dangerouslySetInnerHTML` is a static literal
G1-5	Typecheck gate vacuous — counted `/error TS/`, ignored exit code, OOM'd	🟢 **FIXED**	`typecheck-no-regression.mjs` now treats null status, signal kill, or status > 2 as "did not complete" and fails; the old bug is documented in-file at :93–:101
G1-6	AnA attach button discarded every file	🟢 **FIXED**	Behaviour removed; `client/src/concept2cure/v2/__tests__/anaRailAttach.test.tsx` is a regression test for it
G1-7	~85 of 96 surfaces reachable only by typed URL	🟡 **MUCH IMPROVED**	`RAIL_PRIMARY` (5 entries) replaced by four rails — `RAIL_CORE` 12, `RAIL_SPECIALIST` 6, `RAIL_EXPLORE` 14, `RAIL_QUICK` 9 = **41 rail-reachable** of 83 registered surfaces; `NAV_HIDDEN` 40 → 29. **WO-9** to finish

Five of seven fixed, two improved. This is the strongest evidence in the document that the team's remediation loop works, and it is precisely what a clean-room snapshot cannot see.

---

7. Work orders

Twelve work orders, in `docs/work-orders/`. **WO-0 comes before all of them** — until the branch is green there is no signal to work against. WO-1 and WO-2 are hard prerequisites for WO-3; WO-3 is a hard prerequisite for putting real customer

data in front of anyone.

ID	TITLE	BLOCKS	EXIT CRITERION
[**WO-0**](../work-orders/WO-0-restore-green-canonical-branch.md)	**Restore a green canonical branch**	**everything**	✅ **DONE 2026-09-10** — all six green; sweep non-zero 28 → 23, remainder all `:strict`/unwired/nightly/ENV-BLOCKED
[WO-1](../work-orders/WO-1-schema-authority.md)	Establish schema authority	G1+	`ci:duplicate-table-ddl:strict` and `ci:migration-prefix-collisions:strict` pass with baselines **deleted**
[WO-2](../work-orders/WO-2-blank-database-completeness.md)	Make a blank database complete	G1+	`ci:tables-live-schema` passes with baseline deleted, against a from-scratch install
[WO-3](../work-orders/WO-3-tenant-isolation-proof.md)	Prove tenant isolation on real data	G1+	Live two-tenant probe, plus `requestdb-coverage` 229 → 0
[WO-4](../work-orders/WO-4-enforce-strict-gates.md)	Enforce the six unenforced strict gates	all	Each runs in `pr-checks.yml`, verified by making one fail
[WO-5](../work-orders/WO-5-baseline-governance.md)	Baseline governance and honest gate output	all	All 43 baselines carry owner/reason/expiry; CI prints `RATCHET PASS — N REMAIN`
[WO-6](../work-orders/WO-6-ai-gateway-bypass-burndown.md)	Burn down the 19 AI-gateway bypasses	G1+	`gateway-bypass` baseline 19 → 0, or each survivor re-justified
[WO-7](../work-orders/WO-7-esignature-enforcement.md)	E-signature on regulated promotion	G3, partially G1+	Governed transition refuses without signature manifestation; covered by a test
[WO-8](../work-orders/WO-8-skipped-tests.md)	Triage 34 skipped tests	G1	Each un-skipped or annotated with why it cannot run
[WO-9](../work-orders/WO-9-pilot-surface-lock.md)	Lock the pilot surface set	G1	Pilot surfaces in a rail; the rest behind an explicit experimental affordance
[WO-10](../work-orders/WO-10-deletion-program.md)	Proof-gated deletion program	none — hygiene	Deletion-proof procedure exists **before** anything is deleted
[WO-12](../work-orders/WO-12-complexity-refactor.md)	Complexity growth now inside the eslint baseline	none — deferred	`complexity` ≤ 1,687 and `max-lines-per-function` ≤ 1,190
[**WO-13**](../work-orders/WO-13-grdhe-tenant-scoping.md)	**Give the audit trails a tenant key**	**G1+**	`electronic_signatures` carries a tenant column and is `tenant`-scoped rather than refused; `regulatory_harmonization.audit_log` and `audit.tamper_proof_log` each have a key, a policy, **or** a written exemption naming the cross-org reader that needs one

A withdrawn finding, kept here because the retraction is the useful part. While executing WO-0 I reported that `server/routes/tenant-config.ts` enforced no role check on three mutating routes, having read the middleware chain (`authMiddleware, requireOrganizationContext`) and confirmed that `requireOrganizationContext` checks no role. That was wrong. All three handlers perform the check inline — `if (req.userRole !== 'super_admin' &&`

`req.userRole !== 'admin') return res.status(403)` at :179, :248 and :366. The docblock's claim is enforced; the unused `requireAdminRole` import was an unused import, nothing more.

The error is worth recording because it is the exact failure mode `docs/audit-2026-07/12-findings-register.md` warns about — of ten headline findings that audit put through adversarial verification, six were overstated, three materially. A middleware chain is not an authorization boundary on its own, and a finding that stops at the route signature has not been verified.

Sequencing

```
Days 1-4 WO-0 ← the branch is red; nothing else has a signal until this lands Week 1-2 WO-4 ─┐ (cheap; stops the next regression) WO-1 ─┼─ schema authority Week 2-4 WO-2 ─┘ Week 3-5 WO-3 (needs WO-1 + WO-2 complete) Week 3-5 WO-13 (the other half of the grdhe fix — do not let this sit) Week 4-6 WO-6, WO-9, WO-8 Week 5-7 WO-5 Later WO-7 (G3), WO-10 (hygiene, never urgent)
```

Start with WO-0, then WO-4. WO-0 because a red branch means no work order below it can be measured. WO-4 because it is the cheapest item on the list and it is the reason the others can regress while you work on them.

Two things explicitly *not* recommended

- **Do not bulk-delete the 98 unreferenced modules or act on "505 orphan

endpoints of 884."** A 57% orphan rate in a system in daily use is a detector-precision artefact — the consumer heuristic does not see dynamic dispatch. `unreferenced-modules` sits *at* its baseline of 98, which is a ratchet holding steady, not a fire. WO-10 requires a deletion-proof procedure before a single file is removed.

- **Do not rewrite baselines to make gates green.** Every baseline in this

repository is framed by its own header as a defect list to shrink (`unbacked-tables-baseline.json`: *"Each is a defect awaiting a code-derived migration or a retirement, NOT an approved pattern. Shrink this file; never grow it."*). WO-1 and WO-2 require baselines **deleted**, not rewritten.

---

8. What would change this verdict

The verdict is 🔴 for a real-data pilot on three specific, falsifiable claims. Each has a command that would overturn it:

CLAIM	COMMAND THAT WOULD DISPROVE IT
The deployed schema is not derivable from the repository	`ci:duplicate-table-ddl:strict` passing with no baseline file
A blank database does not provision what the server queries	`ci:tables-live-schema` passing with no baseline, against a fresh install
Tenant isolation is asserted, not proven	A live two-tenant probe in CI, plus `requestdb-coverage` at 0

If those three go green, the real-data pilot bar is met and the remaining work orders are quality, not safety.

One caveat on that sentence, added after §1's two live defects. All three commands above are static. Neither of the two most serious findings in this evaluation would have been caught by any of them going green, because both sat outside what the gates model — a cross-tenant read whose query *did* carry a predicate, and a replayed `INSERT` into a regulated table. So treat the three as necessary and not sufficient. The sufficient test is WO-3's live two-tenant probe: two real tenants, real records, and an attempt to read across the boundary on every route that serves regulated data. Until that has been run and has failed to find anything, "isolated" remains a claim about the code rather than an observation of the system.